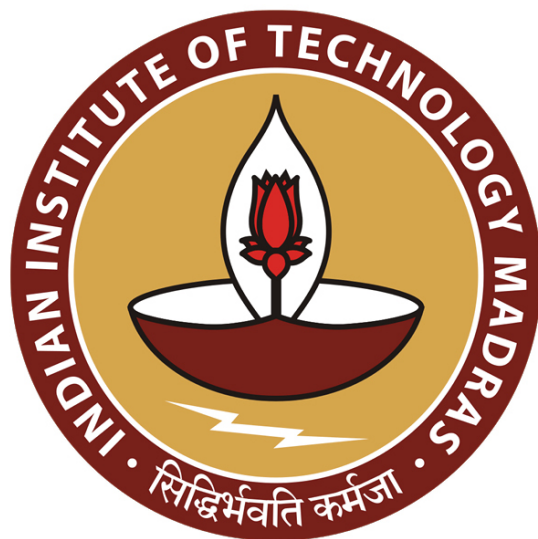


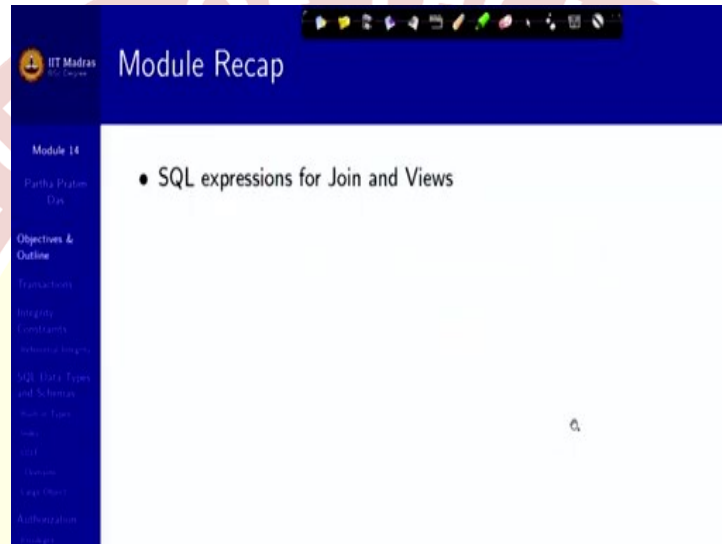


IIT Madras
ONLINE DEGREE

Database Management Systems
Professor. Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Intermediate SQL/3

Welcome to module 14 of Database Management Systems in IIT Madras Online B.Sc. Program.

(Refer Slide Time: 0:27)



We have been discussing about intermediate level features of SQL after having done the introductory ones and in the last module we have discussed about two very important features; join, various types of join, particularly inner and outer join, natural join and so on and we have talked about views, which is also a very important feature by which you can make a virtual relation out of one or more actual relation or other views as well and give that kind of views to users for restricted use.

(Refer Slide Time: 1:12)

IIT Madras

Module Objectives

Module 14

- To understand Transactions
- To learn SQL expressions for Integrity Constraints
- To understand more Data Types in SQL
- To understand Authorization in SQL

IIT Madras

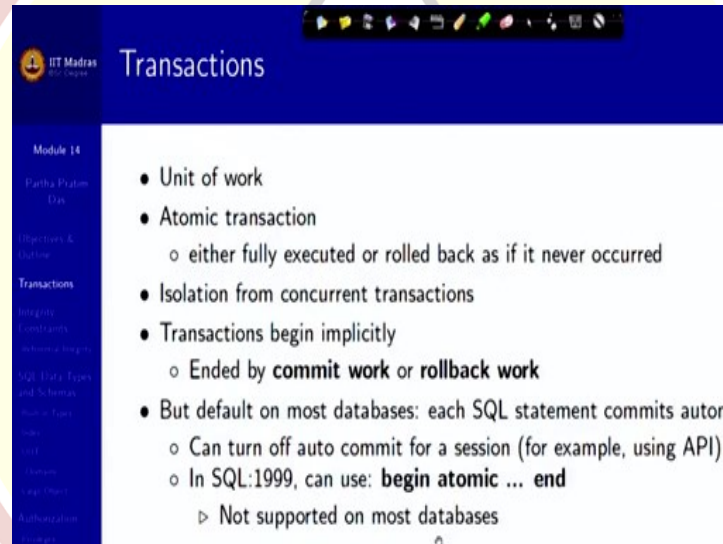
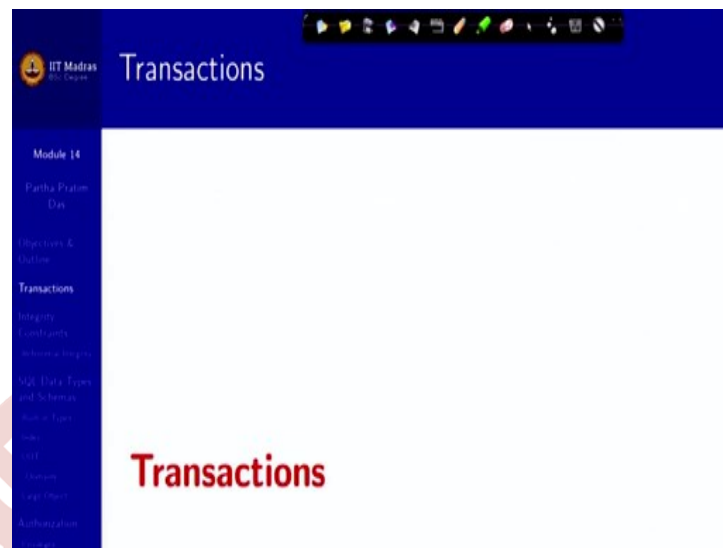
Module Outline

Module 14

- Transactions
- Integrity Constraints
- SQL Data Types and Schemas
- Authorization

Now, we will, today in this module we will cover more of intermediate features, transactions, integrity constraints. We have talked about some data types, we will talk about some more and the authorization aspects where each one of them very critical in their own way. So, this is what will be our main target to learn, so that we can actually get close to using SQL for database applications.

(Refer Slide Time: 1:38)



So, transactions we have mentioned earlier. Just to formally specify and obviously there will be lot more of discussions on transactions that will come later on is, transaction is a unit of work, some unit that we want to do and it is defines to be atomic, transactions are atomic in the sense that if they have one or more SQL statements and other things to execute as a part of doing that job which could be transferring some funds from one account to the other, enrolling a student, registering for a course, whatever.

It is atomic in the sense that either the entire thing is fully correctly executed or it is rolled back if some error, some problem happens in the middle. So, that is a very critical property of the transaction, which is required to keep the database in a consistent state all the time. It will

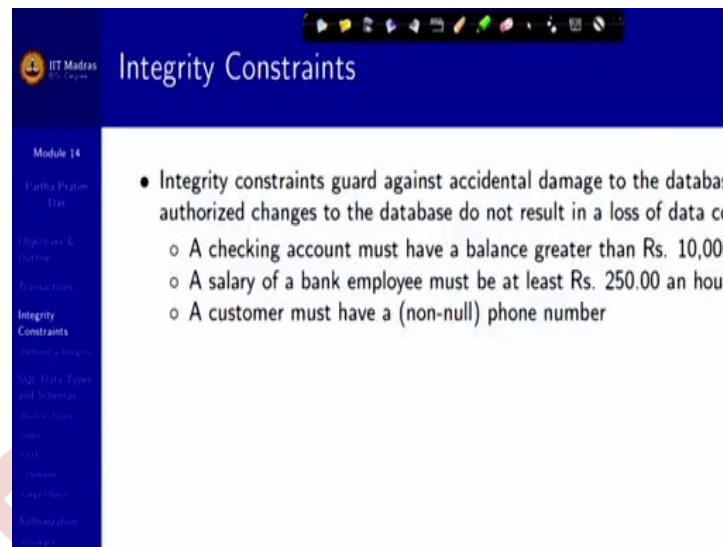
also have to have isolation from concurrent transactions because many transactions will continue to happen on the same tables at the same time.

So, the result of one should not affect the result of the other. We will talk about isolation lot more later on and transactions will begin implicitly, but is ended explicitly by either of committing the work with a commit or rolling back the work, roll back, that is, there was some error so you have to undo whatever you have done and go back to a previous consistent state. Many databases do the commit by default, but it can be turned off as well.

So, many databases do commit by default, but the rollback obviously will be explicit based on if something has happened. So, transactions are a very key area and we will have a separate set of modules to discuss about various aspects of the transaction and how to ensure atomicity and isolation, the acid properties and so on as they are called.

(Refer Slide Time: 3:52)





IIT Madras
Integrity Constraints

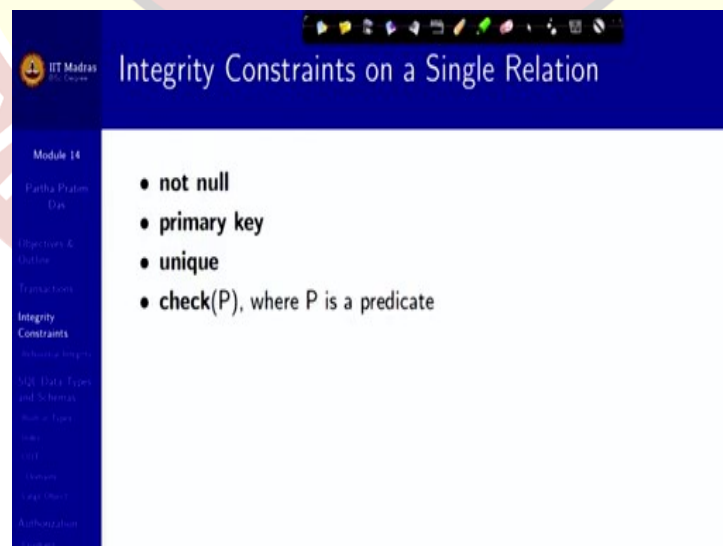
Module 14

- Integrity constraints guard against accidental damage to the database; authorized changes to the database do not result in a loss of data consistency
 - A checking account must have a balance greater than Rs. 10,000
 - A salary of a bank employee must be at least Rs. 250.00 an hour
 - A customer must have a (non-null) phone number

The second is about integrity constraints, we talked quite a bit of that in part of the database design aspect and there is some more that we would like to add here, so it is, these are required to guard against certain basic rules of the real world or, and or I should say, and or certain conditions of the database that we want to preserve and not want to get tampered.

So, real world situations could be that a checking account must always have a balance, a minimum balance or a salary of an employee has to be a minimum wage per hour and so on so forth, some phone you may insist that customers must have non-null phone number and so on, and there could be several other integrity constraints for the database itself.

(Refer Slide Time: 4:48)



IIT Madras
Integrity Constraints on a Single Relation

Module 14

- not null
- primary key
- unique
- check(P), where P is a predicate

So, the typical integrity constraints on a single relation involve something not being null, some attribute not being null, something being a primary key or uniqueness or check clause, which checks for a predicate, so some of these you have already seen.

(Refer Slide Time: 5:05)

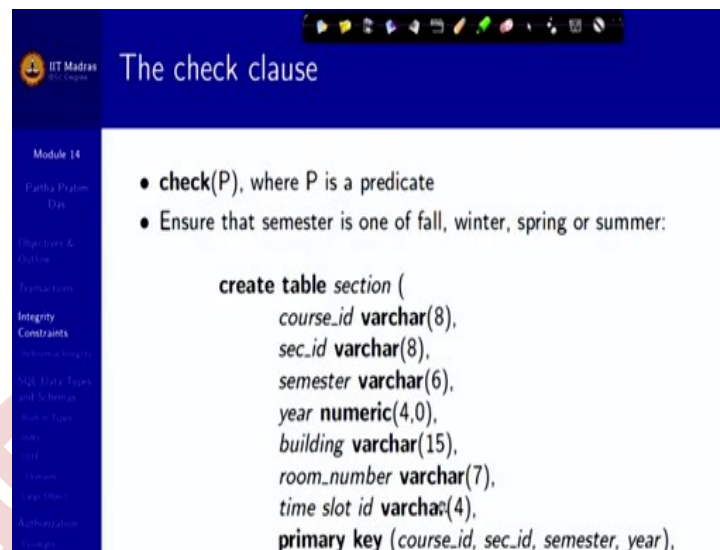
Not Null and Unique Constraints

- **not null**
 - Declare *name* and *budget* to be **not null**
name **varchar(20) not null**
budget **numeric(12,2) not null**
- **unique ($A_1, A_2, \dots, A_m$)**
 - The unique specification states that the attributes $A_1, A_2, \dots, A_m$ form a **key**
 - Candidate keys are permitted to be null (in contrast to primary keys)

Not-null you have seen that you can declare a particular attribute as not null in that case the null value will not be allowed. You can also declare a set of attributes as unique, if you define a collection of attributes as unique that means that in the set of tuples that can go in that table this combination of values on these attributes will never be repeated, so they will not be duplicate, so kind of this is the definition of a, as you know the definition of a candidate key.

But this is a different from a primary key because the candidate key is permitted to be null, whereas primary key cannot be permitted to be null, so this is a uniqueness constraint.

(Refer Slide Time: 5:55)



The check clause

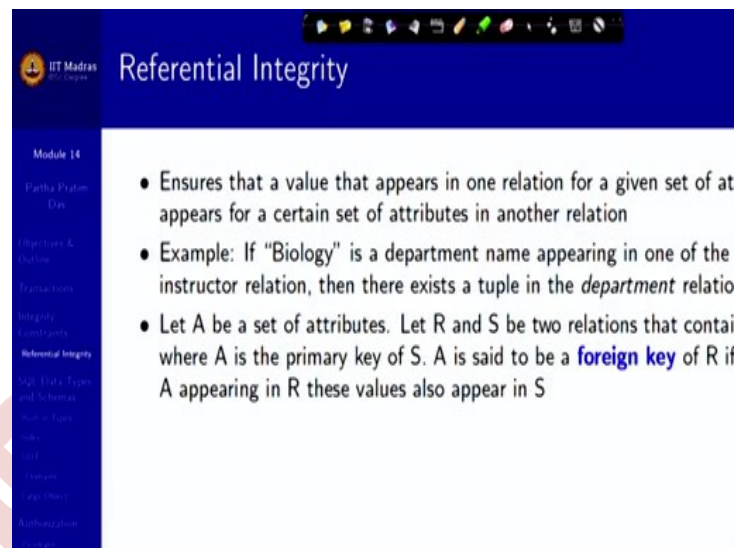
- **check(P)**, where P is a predicate
- Ensure that semester is one of fall, winter, spring or summer:

```
create table section (  
    course_id varchar(8),  
    sec_id varchar(8),  
    semester varchar(6),  
    year numeric(4,0),  
    building varchar(15),  
    room_number varchar(7),  
    time slot id varchar(4),  
    primary key (course_id, sec_id, semester, year),
```

Check is something which you can additionally put to enforce certain business logic, for example, here we have the relation for the section table, in that we have in our university database and several fields are defined, the primary key is defined as a combination of course ID, section ID, semester and year and we need that semester cannot be any arbitrary value, it has to be fall, winter, spring or summer, let us say, I mean, if these are the names.

It could be autumn spring, if you talk about India; it is primarily autumn and spring that is out. So, the value of semester is constrained to be one of these four, so check actually does that, so semester in, in you already know, it is a set membership. In these four possible values says that if you try to set semester as any other value, then this integrity constraint will be violated and that kind of an insertion or update will not be allowed. So, it maintains the integrity of the possible values in the field.

(Refer Slide Time: 7:05)



Referential Integrity

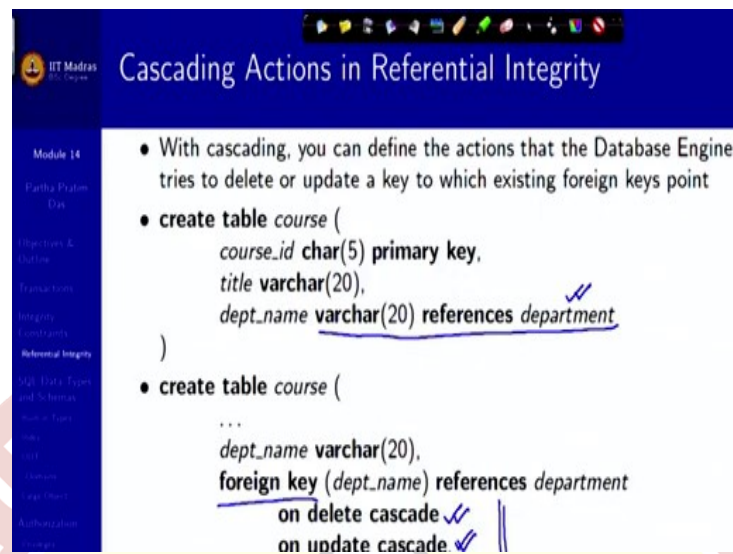
- Ensures that a value that appears in one relation for a given set of attributes appears for a certain set of attributes in another relation
- Example: If "Biology" is a department name appearing in one of the instructor relation, then there exists a tuple in the *department* relation
- Let A be a set of attributes. Let R and S be two relations that contain A. A is said to be a **foreign key** of R if A appearing in R these values also appear in S

We have talked about referential integrity earlier; this is the integrity between two different tables, two or more different tables. So, suppose I have in my instructor table I have an instructor whose department is biology, then it is necessary that the department table has an entry for biology. So, but department table is not a part of the instructor table; but it has a value in the department name as biology.

So, I want that the department table must have an entry tuple where department name is biology, so that we can know the other details of that department from the department table for this instructor and conceptually it is good, I mean, if somebody has an affiliation to a department and if that department does not exist in the department table that describes the set of departments, then it is inconsistent.

So, this is the concept of the foreign key that we had, so the department in instructor becomes a foreign key from the department table, the department name becomes.

(Refer Slide Time: 8:23)



Cascading Actions in Referential Integrity

- With cascading, you can define the actions that the Database Engine tries to delete or update a key to which existing foreign keys point
- **create table course (**
 course_id char(5) primary key,
 title varchar(20),
 dept_name varchar(20) references department
)
- **create table course (**
 ...
 dept_name varchar(20),
 foreign key (dept_name) references department
 on delete cascade
 on update cascade
)

So, you can see what the consequence of that is, so this is how we define the foreign key that we have seen, that the department name in the table, a references department, so it is a foreign key for the department table. Now the question is when we, these two are independent tables, so they will have changes being made, inserts, deletes, updates being made on the two tables independently.

But that might lead to certain inconsistency. For example, now I have an instructor whose department is biology and department table has a department named biology. Now suppose the department table decides to delete the entry for the department name biology. Now what will, this will become inconsistent. So, this foreign key constraint has to further ensure that what should be the policy at that.

For example, I can say that the policy is delete on cascade, on delete cascade, which mean that if the department biology is removed from the department table, then the cascading effect, the chain effect is that all tuples in course which has biology as a department name value will also have to be deleted. Similarly, I can have on update cascade that name of the department biology changes to bioscience.

So, that change has to be reflected in the course, entries in the courses table, the database should take care of this, so this foreign key with the cascade integrity constraints will make sure the proper referential integrity is maintained. There are other options as well for this like no action, setting to null, setting to default and so on. So, this is a very-very important integrity constraint in the design.

(Refer Slide Time: 10:30)

IIT Madras
Module 14
Part 1: Placing Data
Operations & Query
Transactions
Integrity Constraints
Referential Integrity
SQL Data Types and Schemas
Views & Triggers
Security
Backup and Recovery

Integrity Constraint Violation During Transactions

- create table *person* (
 ID char(10),
 name char(40),
 mother char(10),
 father char(10),
 primary key *ID*,
 foreign key *father* references *person*,
 foreign key *mother* references *person*)
- How to insert a tuple without causing constraint violation?
 - Insert father and mother of a person before inserting person
 - OR, Set father and mother to null initially, update after inserting possible if father and mother attributes declared to be **not null**

Then, there could be certain integrity violation during transaction. For example, think about this person database where you have persons, their ID name and there are fields mother and father, so and you have a referential integrity foreign key on father and mother, very interestingly the reference, the referential integrity is with the same relation person.

So, it says, what does it mean? It means that if I am entering the record of a person, then whoever is mentioned as mother or whoever is mentioned as father must already exist in that table. So, how do you do an insertion without causing a constraint violation? One way would be that insert the father and mother of a person before inserting that person, that is one possibility. Other is you could set them as null, if these fields are not null.

Now both of these have, but if they are null you cannot do that, but even if you take this first strategy of inserting the father and mother of a person before inserting that person, then that chain will continue, the father and mother individually, when they are entered, their father and mother will also have to be entered, so somewhere the chain has to end, so if null is not allowed, then somewhere you will have to stop that chain and take care of that in the design.

The other option is to defer the constraint checking, there are possible, I mean there are strategies of doing that, but we will talk about that later on in a later stage.

(Refer Slide Time: 12:18)

The top slide, titled "SQL Data Types and Schemas", shows a sidebar with the following items: Module 14, Part 1: Primitives, Datas, Objects & Tables, Functions, Integrity Constraints, Advanced Topics, SQL Data Types and Schemas (selected), Built-in Types, Views, Joins, Aggregates, and Subqueries. The main content area is currently blank.

The bottom slide, titled "Built-in Data Types in SQL", lists the following data types with examples:

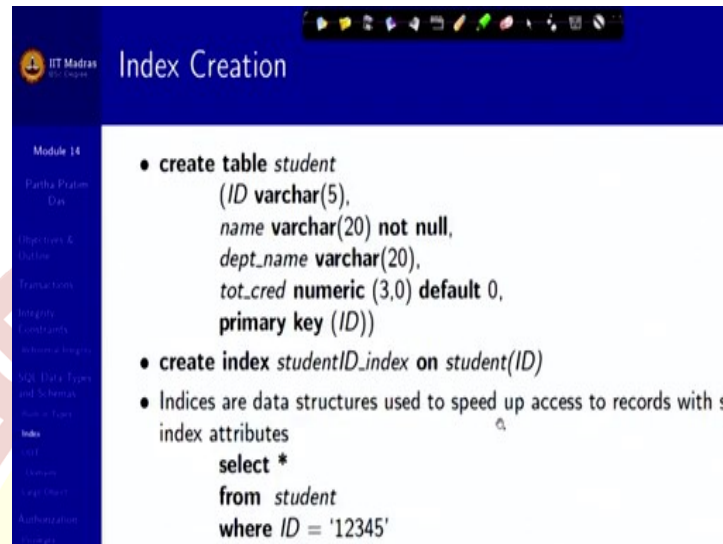
- **date**: Dates, containing a (4 digit) year, month and date
 - Example: **date** '2005-7-27'
- **time**: Time of day, in hours, minutes and seconds.
 - Example: **time** '09:00:30' **time** '09:00:30.75'
- **timestamp**: date plus time of day
 - Example: **timestamp** '2005-7-27 09:00:30.75'
- **interval**: period of time
 - Example: **interval** '1' day
 - Subtracting a date/time/timestamp value from another gives an i
 - Interval values can be added to date/time/timestamp values

The next that we would like to take you through are SQL data types and schemas, you already know a lot of data types like numeric, like integer and like care, where care and so on, but there are certain apparently composite data types which are built-in in SQL, the most important ones are the date, time data types, you can specify a date with a certain format, there are number of formats that you can choose.

You can specify a time or you can specify a time stamp which is date and time together and any difference between a date and a time stamp and so on or between two dates and so on is called an interval period of time, so interval could be one day or any other and interval could be added or subtracted from the date time data. So, it is very easy convenient for date time

manipulation because that is something which you need, I mean, almost everywhere in every database application design.

(Refer Slide Time: 13:23)



The slide is titled "Index Creation" and is part of "Module 14" from IIT Madras. It contains the following content:

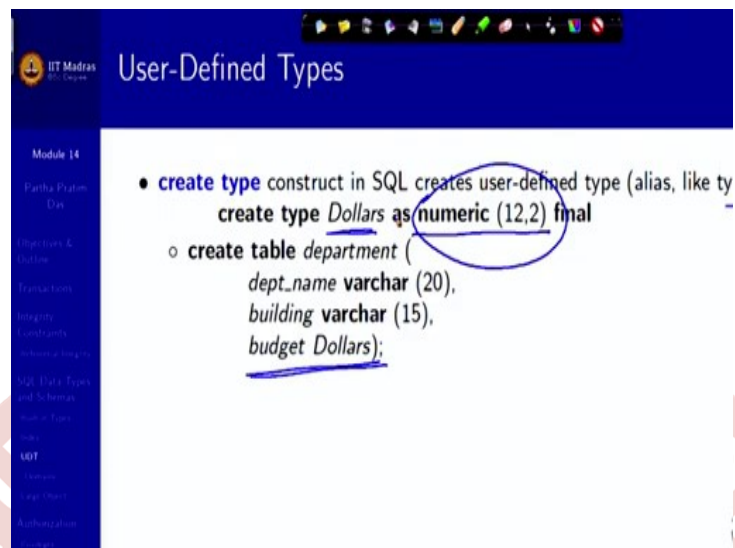
- **create table student**
 (*ID* varchar(5),
 name varchar(20) not null,
 dept_name varchar(20),
 tot_cred numeric (3,0) default 0,
 primary key (ID))
- **create index studentID_index on student(ID)**
- Indices are data structures used to speed up access to records with sp
 index attributes
 select *
 from student
 where ID = '12345'

Another very important type of data that you have to create is called an index. Index is a very deep concept and will come to that but just to introduce you to the datatype, so here is a student's table where you have ID varchar 5 as a primary key, so that is a unique key. So, it is, in that condition it is possible to create an index that is the way to do it, give it a name, create ID index on the student table using the attribute ID.

So, these are, indices are separate data structure that are created if you have idea about B tree and so on, you will somewhat understand the value of indices, this help to speed up the database record processing, basically to speed up the process, I mean, search of any particular or a set of records, for example, if you, since ID is a primary key, if you look for a student with a given ID you will get only one answer.

Without the index it will need that the database table has to go through all the IDs and match one, with the index, it could be, so that is log n time, with index, so with this additional data structure you may be able to do it in log n or even much lower than even log n where there are n number of tuples in the table at that point of time. How we do all that and how to maintain index? We will talk about that separately, but just that the indices do provide additional type of data type in SQL.

(Refer Slide Time: 15:12)



User-Defined Types

- **create type** construct in SQL creates user-defined type (alias, like ty

```
create type Dollars as numeric (12,2) final
```

- **create table** department (
dept_name varchar (20),
building varchar (15),
budget Dollars);

Another is user defined data type, which are in a true sense they are not that way user defined, but they are more like aliases that you can create, convenient aliases with certain properties that you can create, like you can, if you know C, C plus plus you will know about typedef, you can give a certain data definition a name. So, we say create type, I am giving a name dollar as numeric 12, 2 and then I can use that in defining the type of attributes.

The most useful reason for doing this is to make sure that there is better semantic understanding for it, if you just had budget numeric 12, 2, it is difficult to understand the currency, whether it is in euro, in dollar, in rupees or in peso, or something else. So, by giving this name you are actually making it easier to understand, easier to manage but it is not, user defined types are not limited to just a semantic understanding or readability, they are extended into what are called domains. So, domains are again user defined types UDTs, but domains can also add some constraint.

(Refer Slide Time: 16:34)

Domains

- **create domain** construct in SQL-92 creates user-defined domain types
`create domain person_name char(20) not null`
- Types and domains are similar
- Domains can have constraints, such as **not null**, specified on them
`create domain degree_level varchar(10)`
`constraint degree_level_test`
`check (value in ('Bachelors', 'Masters', 'Doctorate')):`

So, here you can see that we are doing a create domain of a person name, we are not just up to this point it is just a simple UDT, but what we have added, which is necessary, for which we need the domain is that not null, so which means that if I say that any person name, name, person name, then I will get two information, one is it is a varchar 20 and it will, I will not explicitly have to say it is not null.

There are so many things which are very commonly used in this way, like name pattern for different things you can have a common domain for that. So, it can also have like a check constraint, so you can say degree level, varchar 10 and the constraint that you are putting on this is that the degree has to be one of these three.

So, domains become very useful in creating constrained data types in SQL and give it a name which can be consistently used all across your database design, this naturally, not only makes it semantically clear, but makes it very convenient for the programming, so you can think of, these are like reusable data types.

(Refer Slide Time: 18:04)

Large-Object Types

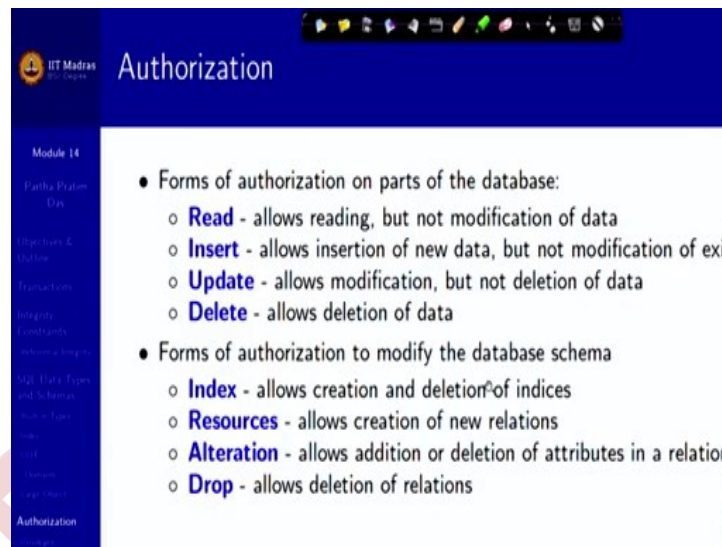
- Large objects (photos, videos, CAD files, etc.) are stored as a *large object*
 - **blob**: binary large object – object is a large collection of uninterpreted data (whose interpretation is left to an application outside of the database)
 - **clob**: character large object – object is a large collection of character data
 - When a query returns a large object, a pointer is returned rather than the object itself

There are some other professions of data types which are called blob: binary large object or clob: cloud character large object, which are used to keep different forms of binary or character data, for example, images, videos, and so on. Those are not stored as a part of the database; they will be part of the file system usually.

But a reference to them will be maintained, so when you access a field which has a blob, you do not get the data, but you get a pointer to the data. So, because they are, so in that way any unstructured data also can be maintained in the database. This is the large object type properties.

(Refer Slide Time: 18:46)

Authorization



Authorization

Module 14

- Forms of authorization on parts of the database:
 - **Read** - allows reading, but not modification of data
 - **Insert** - allows insertion of new data, but not modification of exis
 - **Update** - allows modification, but not deletion of data
 - **Delete** - allows deletion of data
- Forms of authorization to modify the database schema
 - **Index** - allows creation and deletion of indices
 - **Resources** - allows creation of new relations
 - **Alteration** - allows addition or deletion of attributes in a relation
 - **Drop** - allows deletion of relations

Finally authorization, obviously it is very important to ensure that not everybody can do and change the database or see everything, so there are different forms of authorization, which apply to different parts of the database or different operations of the database, like some of the very common authorizations are read, insert, delete and update. That is basically read write authorization.

So, someone who is trying to read must have a read permission, read authorization, someone who is trying to update a value must have authorization to be able to update, and there could be also authorizations on the operations on database schema, like we just talked about indexing. So, someone who can create an index, can everybody keep on creating index, there is a cost of creating and maintaining index also.

It is, index may not change the data, but index may very significantly affect the performance of the database, so you have to constrain as to who can create the index, who can create new relations, like who can do create table and this kind of things, who can alter relations adding or deleting attributes, who can drop relations.

So, all schema based operations, so in SQL we have seen variety of different operations and for a majority of them or for a large part of them depending on the type of operation, there are separate authorizations that are required.

(Refer Slide Time: 20:22)

Authorization Specification in SQL

- The **grant** statement is used to confer authorization
grant <privilege list>
on <relation name or view name> **to** <user list>
- <user list> is:
 - a user-id
 - **public**, which allows all valid users the privilege granted
 - A role (more on this later)
- Granting a privilege on a view does not imply granting any privileges relations
- The grantor of the privilege must already hold the privilege on the sp the database administrator)

So, you provide authorization by what we say are privileges, privilege is something which is, which gives you the authority to do something. So, you grant a people privilege on a relation to a list of users. So, that is what is typically the use of authorization.

(Refer Slide Time: 20:53)

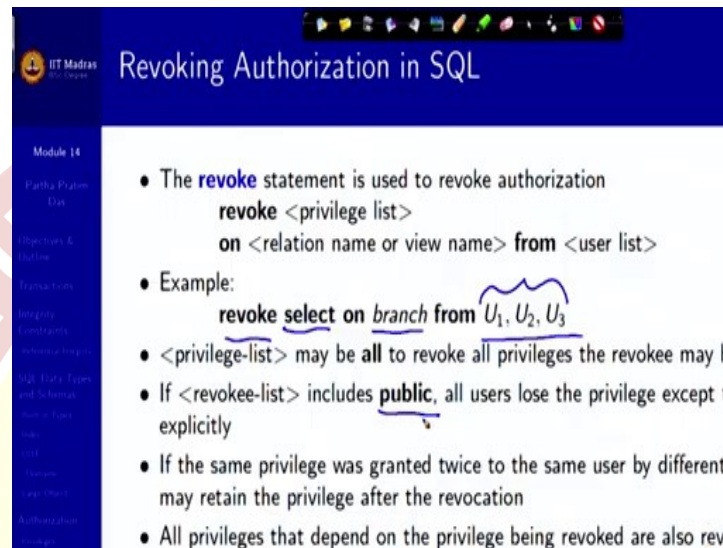
Privileges in SQL

- **select**: allows read access to relation, or the ability to query using the
◦ Example: grant users U_1 , U_2 , and U_3 **select** authorization on the
grant select on instructor to U_1 , U_2 , U_3
- **insert**: the ability to insert tuples
- **update**: the ability to update using the SQL update statement
- **delete**: the ability to delete tuples.
- **all privileges**: used as a short form for all the allowable privileges

So, let us see some example, so suppose we have a privilege called select. What does that mean? This basically is a read authorization, by select what I can do; I can read the data from one or more relations. So, when I have that read authorization given through the select I will put it as grant that is the basic command, then what is the privilege is select, then what is the table on which I am giving it, that is instructor, then whom I am giving it, there is a list of users, it could be public also.

If it is public then everybody can do it, but usually it will not be that, it could be a roles also, we will come to that. So, similarly there are privileges like insert, privilege for insert, so grant insert on and so on, grant delete on and so on, or you could give all privileges, by which you will mean that you will have a privilege to do all of these.

(Refer Slide Time: 22:04)



Revoking Authorization in SQL

- The **revoke** statement is used to revoke authorization
revoke <privilege list>
on <relation name or view name> **from** <user list>
- Example:
revoke select on branch from U₁, U₂, U₃
- <privilege-list> may be **all** to revoke all privileges the revokee may have
- If <revokee-list> includes **public**, all users lose the privilege except the grantor explicitly
- If the same privilege was granted twice to the same user by different users, the user may retain the privilege after the revocation
- All privileges that depend on the privilege being revoked are also revoked

Now naturally if you can grant a privilege, you can also revoke a privilege that is authorization can be granted as well as revoked. So, when you revoke that is you cancel that, you say no, this person is no more working in the organization, this person has no authorization to access the database or this person is not working in this project.

So, the person may have authorization to access 10 other tables, but for these two tables the authorization is withdrawn. So, the revocation in the same way will have a privilege list, revoke, what are you revoking, the privilege to select. What are you, on which relation that is a branch and who are the users for which the revocation is happening?

So, the privileged list may be all, if you want to revoke all privileges, the revokee list, this is called the revokee list, the users whose authorization you are revoking includes public, then all users will lose the privilege to operate except the person who is granting it. Now, obviously the person who is granting a privilege or revoking a privilege must have that privilege and also the privilege to grant privileges.

Now, otherwise what would have happened? I would have gone in, I can grant privilege to myself and be happy that should also not be possible. So, there is a special privilege which allows you to give privileges to others or authorization to others.

(Refer Slide Time: 24:02)

Roles

- create role instructor;
grant instructor to Amit;
- Privileges can be granted to roles:
grant select on takes to instructor;
- Roles can be granted to users, as well as to other roles
create role teaching_assistant
grant teaching_assistant to instructor;
 - Instructor inherits all privileges of teaching_assistant
- Chain of roles
 - create role dean; _____
 - grant instructor to dean; _____

You can define roles for authorization, roles are like that instead of, I mean, suppose there is a development group, there are 20 engineers in the development group. Now as a group of developers they need a certain type of authorization. Now instead of giving each and every granting, each and every privilege to each one, which is explosive, you can say that what is the role, the role of this person is to develop so this person needs this, this, this, this privileges.

So, instead of granting it to that person specifically you create a role, give it a name you create a role and grant that the privilege to that role, so you are doing grant select on the relation takes and here you are not granting it to an individual user, but you are actually granting it to a role. Now, how do you add a person to the role that is grant instructor to?

Now Amit is a particular user, so when you say grant instructed to Amit and then you grant certain privileges to the instructor role, Amit by inclusion gets those privileges, so that is the basic property of the role. And roles can be granted to user, for example, you have a role, teaching assistant and you grant teaching assistant to instructor.

So, which means the instructor now get all the privileges of the role teaching assistant, kind of becomes a superset, not the other way. So, this is, in this way you can have a hierarchy of roles and according to the role that a person is playing he or she will get all the privileges that are granted to that particular role.

So, you can have a create role dean, grant instructor to dean, then grant dean to Satoshi, so which means that Satoshi will have all privileges of the dean's role which has all privileges of

the instructor role. So, anything that you have granted to instructor will be available to dean, anything that you have specially granted to the dean role, will also be available to Satoshi, but will not be available to others who have just instructor role, but not the dean role. So, that is the basic property of the roles.

(Refer Slide Time: 26:49)

Authorization on Views

Module 14
Python Program
Data
Object-Oriented
Database
Transaction
Security
Views
Authorization

- **create view geo_instructor as**
(select *
from instructor
where dept_name = 'Geology');
- **grant select on geo_instructor to geo_staff**
- Suppose that a geo_staff member issues
select *
from geo_instructor;
- What if
 - geo_staff does not have permissions on instructor?
 - creator of view did not have some permissions on instructor?

Authorization can be given specifically on views also, so you can create a view as you can see here, this is the view query and on this geo instructor view that has been created and access select privilege has been given to the geo staff role, so if the geo staff now issues a select statement, so geo staff will be able to do that.

Now the interesting thing is that the geo staff is getting the permission on the view, so the geo staff may not have the same permission of select on instructor, but you will still be able to do the select on the geo instructor view that is the beauty of the, because this part is hidden from the geo staff, that was the property of the view. So, there are some who have, who may have the access select on the instructor table.

The geo staff does not have that but geo staff is given a particular geo instructor view on which he or she has been granted the select privilege, so the person will be able to do the select on the geo instructor, so these are the different nuances that you have to keep in mind.

(Refer Slide Time: 28:27)

Other Authorization Features

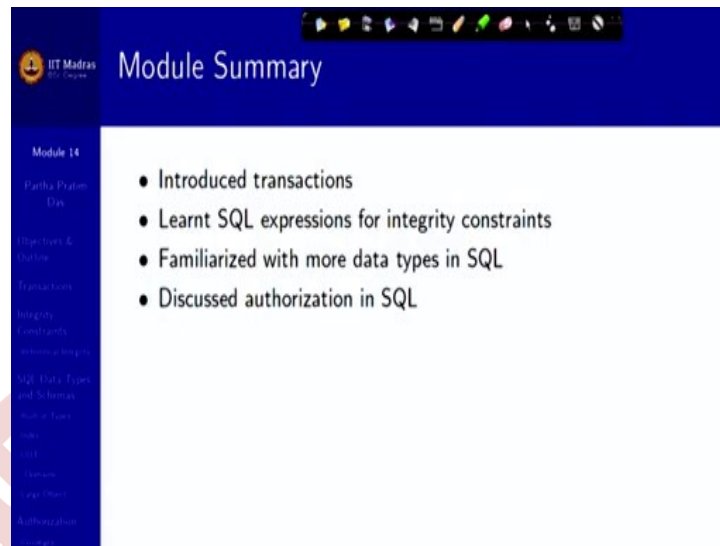
- **references privilege to create foreign key**
`grant reference (dept_name) on department to Mariano;`
 - why is this required?
- **Transfer of privileges**
 - `grant select on department to Amit with grant option;`
 - `revoke select on department from Amit, Satoshi cascade;`
 - `revoke select on department from Amit, Satoshi restrict;`

So, there are several other authorization features also, authorization by itself is a big topic, so you can give reference privilege, the privilege to create foreign key because you are setting in, so you can understand that this is more like a schema based authorization, so you can set a foreign key constraint, referential integrity constraint.

If you have this privilege, which may be important for some users because the user want to maintain a certain consistency in the database which was not originally planned for and finally you can also transfer privileges like you are doing here, grant select department to Amit, this much was, so which says that on department table Amit, the user will have a select authorization and then you are saying with grant option.

With grant option means that Amit now can grant this privilege to someone else, so that is how to say that you can, you will also have to say who can actually, who get authorized to authorize others, so in this way the privilege to authorize who can also be granted to certain users, roles and so on. Similar, things are possible for cascade, for restrict and so on. So, these are revoked examples.

(Refer Slide Time: 30:04)



So, that was about the authorization. So, in this module we have discussed about four very key areas; the transactions, which are the building block of databases; the integrity constraints, which are necessary for consistency, we will see lot more of both of these later on; then certain extra data types and finally the issue of authorization has been discussed and maybe we will try to do some more of that in later modules. Thank you all very much for your attention and see you in the next module.